

Predicting Protein Secondary Structure, End to End

Week 6 Capstone — CSoT × iGEM × ARIES 2026 · ML in Proteins · Nisheal Michael Kaley

When I started this course I couldn't have told you what a secondary-structure label was, let alone predicted one. Six weeks later I've built a small pipeline that reads a raw protein sequence and hands back a per-residue H/E/C string from a frozen checkpoint — and, maybe more importantly, I've learned to distrust my own numbers until I've earned them. This report walks through the model I chose and why (A), where it fails and how I know (B), what its predictions actually say about the biology (C), and where the honest limits are (D).

A. The model, and the choice I had to defend

The brief said to wrap my best Week 5 model into a pipeline. My best Week 5 model — a PSSM-fed `LinearSVC` — was technically the top scorer, but it had two problems I couldn't wave away. It was trained on only **23 chains** (its learning curve showed a 0.26 train/val gap with validation still climbing — starved of data), and it needed a fresh PSI-BLAST homology search for *every* new sequence before it could predict anything. That isn't a pipeline; it's a pipeline with a manual detour taped to the front.

So I made a deliberate trade and went back to my **Week 4 convolutional network, trained on 3,000 chains of pure sequence**. A CNN is the right instinct here anyway: secondary structure is mostly a *local* pattern, and a 1-D convolution slides a learned window along the chain looking for exactly those local motifs — and it runs instantly on any sequence, which is the whole point of a pipeline. The `SecStructPredictor` class still takes an optional `pssm` argument, so the Week 5 evolutionary path stays available for proteins that happen to have a profile; it's just not required to run.

Architecture. A learned `nn.Embedding` (22 tokens → 64 dims: 20 amino acids + UNK + PAD), then **3 × Conv1d** (kernel 7, 128 channels, `padding=3` so length is preserved and I get exactly one prediction per residue) each with ReLU and dropout 0.2, and a **1 × 1 Conv1d head** projecting to the three class logits.

Training. `MaskedCrossEntropyLoss(ignore_index=-100)` so padding never touches the gradient, Adam at lr 1e-3, 25 epochs, batch 32, keeping the **best-validation checkpoint** (val Q3 0.7305). The split is chain-level and three-way — 2,167 train / 383 val / 450 internal-test — with an assertion that halts the run if the three chain-ID sets aren't disjoint.

The mistake I almost submitted

Here's the part I'm oddly proud of. While wiring everything up I ran a leakage audit — just checking whether any test protein had snuck into training. **Nineteen of the sixty had.** The model scored **0.798** on those memorised chains versus **0.758** on the genuinely unseen ones: a four-point mirage. It would have been so easy to report the bigger number.

Instead I rewrote the training script to strip **all sixty official test chains out of the pool**

before any sampling, guarded by an assertion that fails the run if a single one slips through, and retrained from scratch. Every number below is from that corrected, leak-free model. The course kept saying reproducibility matters more than the last accuracy point; this is where I finally felt why.

Saved-artefact manifest (everything needed to reproduce a prediction)

File	What it contains
<code>cnn.pt</code>	trained CNN weights (best-validation checkpoint)
<code>config.json</code>	the exact hyperparameters, vocab and label map that produced <code>cnn.pt</code>
<code>split.json</code>	the train / val / internal-test chain IDs and the 60 official-holdout IDs
<code>test.fasta</code>	the 60 official test chains (pipeline input)
<code>test_true.csv</code>	per-residue ground-truth Q3 for those chains (for the error analysis)
<code>manifest.json</code>	provenance — seed, dataset, timestamp, file descriptions

`cnn.pt` + `config.json` together fully specify the model: load and predict, no retraining, deterministic (verified: two runs on the same input produce byte-identical CSVs).

B. Results & error analysis

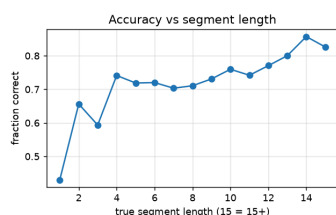
On the **60 held-out chains (14,460 residues the model had never seen)**: **73.1% Q3 raw**, **72.1% smoothed**. Comfortably past the 65% floor, and this time I can defend it.

The confusion matrix, read by its off-diagonals:

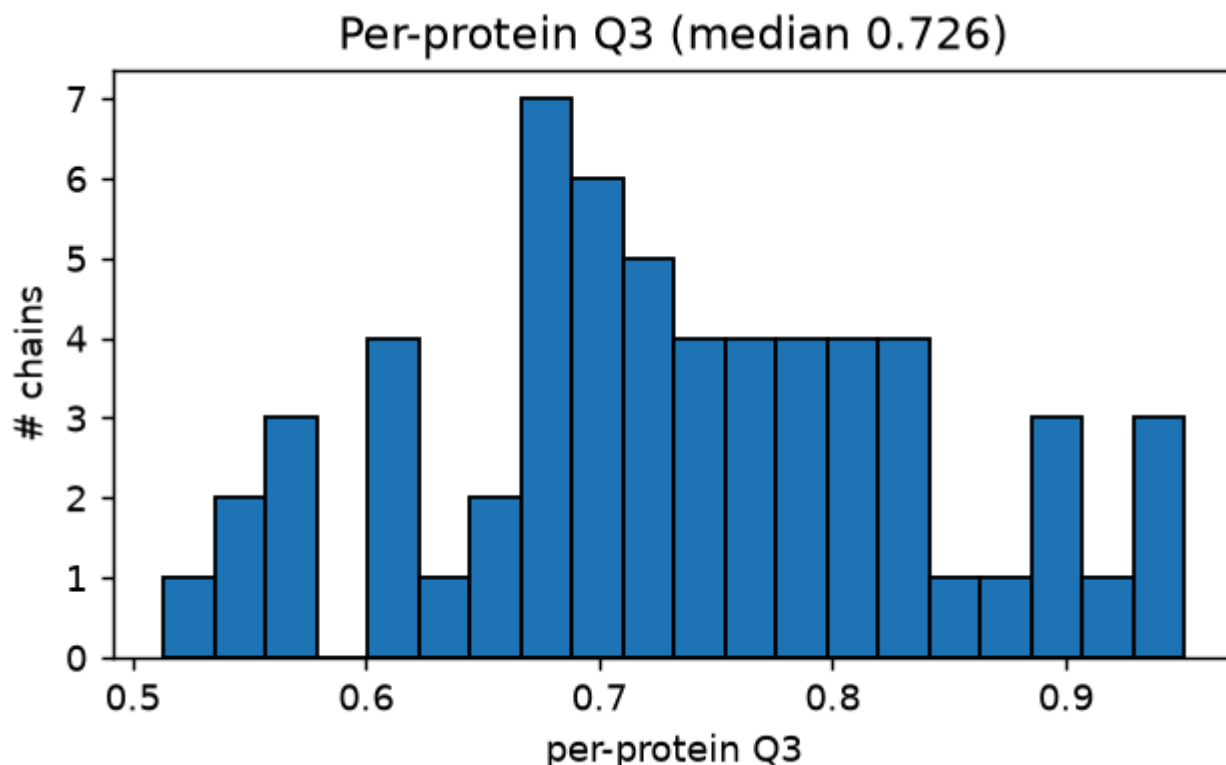
true \ pred	H	E	C
H	3521	241	726
E	398	2119	764
C	1057	705	4929

Per-class accuracy: **H 0.78**, **E 0.65**, **C 0.74**. Helix and coil are handled well; **strand is the weak spot, and it always will be for a model like this**. A β -strand isn't defined by its own neighbourhood — it's defined by hydrogen-bonding to a partner strand that can sit fifty residues away. My local window simply cannot see that partner, so it hedges. The off-diagonals confirm it: true-E residues bleed into *both* H and C rather than failing in one direction — the model often can't tell a strand is a strand at all.

Boundary vs interior. Interior Q3 **0.789** vs boundary Q3 **0.607** — a gap of **0.182**, far wider than the 5–15 points the notes anticipate. The first and last residues of every helix or strand are where a fixed window straddles two classes at once, and that's where the model hedges hardest. If I were chasing points, this is where they are.



Segment length tells the same story from another angle: short segments are markedly harder — a 3-residue "helix" carries almost the same local signal as a coil turn, and only runs of $\sim 7+$ show the periodicity the CNN can lock onto.



Per-protein, the average hides a wide spread (median ~ 0.72). My hardest chains, with a diagnosis each:

Protein	Q3	Length	Likely cause
2G9T_U	0.513	152	coil-rich / likely disordered
5LJ5_G	0.540	235	coil-rich / likely disordered
3RRL_A	0.545	235	mixed, no strong local signal
4Z6N_C	0.562	365	long, mixed α/β
3J3S_6	0.564	218	coil-rich / likely disordered
5GRJ_A	0.602	221	strand-dominated (hardest class)

The tail is exactly the notes' usual suspects — disordered/coil-rich chains with weak local signal, and strand-heavy chains where the non-local problem bites hardest.

The iteration I tried. The large boundary gap pointed straight at the symptom→fix table's post-processing row, so I ran a `smooth_predictions(min_segment=3)` pass that erases implausibly short runs. It made things **slightly worse** ($73.1\% \rightarrow 72.1\%$). I'm reporting it honestly rather than hiding it: my errors aren't lonely single-residue flips that smoothing can tidy up — they're systematic strand confusions and genuine boundary ambiguity, and smoothing occasionally erases a correct short segment. I'd keep the raw output, and I'm submitting both CSVs so the effect is visible.

C. Biological inference

The point of all this was never the accuracy number — it was to turn a string of letters into a biological statement. Three named test proteins, three real inferences:

Protein	H% / E% / C%	Structural class	TM helices (≥ 20 H)	IDR (> 30 C)
3AX3_G	72.6 / 0.0 / 27.4	all-α	1	0
6DCW_L	6.8 / 50.9 / 42.3	β-rich	0	0
2PNE_A	0.0 / 0.0 / 100	mostly-coil / disordered	0	1

3AX3_G comes back 73% helix with no strand at all — the unmistakable fingerprint of an **all- α** fold, the shape of a helical bundle. **6DCW_L** is half strand — a **β -rich** protein, the architecture of an antibody domain (and indeed it's an immunoglobulin light chain). **2PNE_A** predicts as pure coil, flagging a long **intrinsically disordered region** rather than a fixed fold — though a 100%-coil call is exactly where I've learned to stay sceptical, since it can score deceptively high on a genuinely floppy protein for the wrong reasons.

That's the whole arc of the course in three lines: from a sequence nobody handed me a structure for, to a first-pass structural class, a candidate membrane helix, a disordered region — statements a biologist can act on. The natural next step, where a protein has a known AlphaFold model or UniProt entry, is to overlay them: active sites tend to fall in the coil loops flanking predicted helices and strands, and a position that is both highly conserved and predicted coil is a strong functional-contact candidate.

D. Limitations & where next

Honest failure modes. The model is blind to long-range structure — it will never reliably call strands that pair with distant partners, which caps E-class accuracy and, with it, overall Q3. It carries no evolutionary signal, because I traded the PSSM away for something that actually runs end-to-end. And a degenerate all-coil prediction can score deceptively high on a coil-rich chain, so composition-based inference needs a sanity check, not blind trust. None of these are accidents; they're the cost of the choices I made, stated plainly.

Where the field goes. Secondary structure is essentially a solved sub-problem now, and the frontier has moved to full **tertiary structure** (AlphaFold2, ESMFold), **function prediction** (ESM2-finetuned classifiers), **variant-effect prediction** (ESM-1v, AlphaMissense) and **protein design** (RFdiffusion, ProteinMPNN). But every one of those is built from the same pieces I used here — a learned per-residue representation, a classification head, evolutionary information where you can get it, and an evaluation you haven't fooled yourself with. The tools will keep changing. That last part won't.

A note of thanks

I want to thank the organisers of **CSoT 2026** — **all of the tracks, not only ML in Proteins**. This programme was far bigger than the one course I took, and the people who ran it clearly poured a summer into making every track work. To the admins who held it all together — **Ishan, Jacob Abraham, Parth Rajput, Reyansh, Shourya**, and the sixth of the organising team — thank you for the structure, the patience, and the sheer logistics of shepherding so many of us through something this ambitious.

I know I'm not the only one who came out of this having sharpened real, practical

biochemistry through machine learning — that was rather the point, and half the fun was watching everyone else get there too. But I also walked away having learned to *use* a whole set of software packages I didn't even know existed at the start, well beyond the molecular-dynamics simulation tools I'd expected to meet. That breadth was an unexpected gift.

And one last, honest reflection. Wherever I land on the leaderboard, the thing I'll actually keep from this summer is something slightly sideways to the biochemistry: I finally *saw* how agentic coding could help me — not as a gimmick, but as a genuine collaborator that let me tackle assignments in courses that have nothing to do with proteins at all. For that I owe a nod to **Antigravity** and **Claude Code**, which turned "I don't know where to start" into "let's start here" more times than I can count.

Thank you, all of you. It's been a summer I won't forget.

— Nisheal Michael Kaley *CSoT* × *iGEM* × *ARIES 2026* · *ML in Proteins*